

Omar Ahmad Elhakim

Maadi, Cairo, Egypt | +201127002031 | omarelhakim@icloud.com |  

Education

Ain Shams University

Graduated June 2026

BSc in Computer Science, Scientific Computing Department | GPA: 3.3

Experience

Zinad

July 2024 – Present

Software Engineer (Part-Time)

- Built a phishing simulation engine that generates themed emails and working fake landing pages from the parameters a user picks, driving **Ollama** from a **Laravel** backend and rendering the emails with **MJML**.
- Wrote an email analyzer in **Python**, **LangChain** and **Ollama** that flags phishing from tone, context and grammatical anomalies; its verdicts feed the platform's security metrics.
- Developed an internal **RAG** system that answers employee questions from the company's policy PDFs, splitting the documents with **recursive character chunking** with overlap, embedding the chunks into a **FAISS** vector store and retrieving the passages relevant to each question.
- Shipped a chatbot into the live application with **LangChain** and **Ollama**, streaming replies token by token instead of making the user wait for the full answer.

Projects

Neuromarketing: EEG Emotion Recognition — Graduation Project

- Classified emotion from raw **EEG** brain signals on the **SEED-IV** dataset, tuning nine deep-learning architectures (**DaViT**, **DGCNN**, **CCNN**, **BFENet**, **TSception** and others) in **PyTorch** under subject-dependent and cross-subject protocols, reaching **96%** accuracy on binary and **85%** on 4-class emotion recognition.
- Combined the dataset's precomputed **differential entropy** features with our own: per-band differential entropy extracted through **db4 wavelet** decomposition, and hemispheric asymmetry (**DASM**) features derived from it.
- Built the raw-signal preprocessing pipeline in **SciPy** and **NumPy** (Butterworth bandpass, downsampling, z-score normalization, windowing), and re-ran the experiments on the **14 channels** of a consumer Emotiv EPOC+ headset to test viability on affordable hardware.

Machinery Fault Diagnosis from Vibration Signals

- Detected mechanical faults in rotating machinery from raw accelerometer signals as a **multivariate time-series classification** task, training **InceptionTime**, **xResNet1d**, **LSTM-FCN** and custom CNN/RNN models in **PyTorch** and **tsai** to **0.88 F1** on flow faults and **0.81 F1** on bearing faults.
- Wrote the **NumPy** data library behind every experiment, covering eight fault categories, per-fault channel selection, overlapping sliding windows and an optional **FFT** half-spectrum.
- Tuned learning rate, batch size and window size with **Optuna** pruning callbacks, countering heavy class imbalance to optimize **F1** over accuracy.

RAG

- Built a document Q&A service in **FastAPI** and **LangServe**, splitting a company policy corpus into overlapping chunks with a **recursive character splitter**, embedding them with **word2vec**,

indexing them in a **Chroma** vector store, and grounding **Gemini** on the chunks retrieved for each question.

- Added a phishing screener that runs an email body through a self-hosted **Llama** on **Ollama**, returning nine checks (urgency language, requests for confidential information, character-encoding tricks such as Cyrillic lookalikes, and others) as structured **JSON** with English and Arabic justifications, covered by end-to-end API tests.
- Extended retrieval to databases with a **text-to-SQL** chain that feeds the live **MySQL** schema to the model, executes the generated query and phrases the result in plain English.

Blood Cell Detection with YOLO [↗](#)

- Located and classified red cells, white cells and platelets in blood-smear microscope images, fine-tuning pretrained **YOLOv8** and **YOLOv5** detectors on 364 annotated slides and benchmarking the two generations on the same split at **0.93 mAP@50**.
- Wrote the dataset pipeline behind the training run, converting ~5000 corner-coordinate bounding boxes into normalized **YOLO** center format with **OpenCV**.
- Evaluated per class, holding **0.98 mAP@50** on white cells despite being outnumbered ten to one by red cells.

Credit-Card Fraud Detection [↗](#)

- Flagged fraudulent transactions in a log of **1M** card payments of which only **0.57%** were fraud, training **Random Forest** and **SVM** classifiers in **scikit-learn** to **0.94 balanced accuracy** and **89% recall**.
- Engineered transaction features (deviation from the mean amount, hour of day, time since the card's last payment, merchant and category frequency) and used correlation against the fraud label to keep the four that carried signal.
- Compared **SMOTE** oversampling against majority-class undersampling and scored on recall and balanced accuracy.

Multilayer Perceptron from Scratch [↗](#)

- Implemented a neural network in pure **NumPy**, writing feedforward, **backpropagation** and weight updates directly from the derivatives with no deep-learning framework; reached **95%** test accuracy on three-species classification.
- Made the network fully configurable: any depth and width of hidden layers, **sigmoid** or **tanh** activation paired with its matching derivative, and an optional bias term.

ECG Heartbeat Classification [↗](#)

- Separated healthy from abnormal heartbeats in ECG recordings, building the signal pipeline in **SciPy** and **PyWavelets** (4th-order **Butterworth** bandpass, 2-level **db4 wavelet** decomposition, min-max scaling) and classifying 400 held-out beats with **100%** accuracy.
- Reduced each 300-sample beat to its level-2 wavelet coefficients, which separate the two beat shapes well enough for a **5-NN** classifier to reach that accuracy without tuning.

Road Segmentation with Naive Bayes [↗](#)

- Segmented the road surface out of driving-scene photos pixel by pixel with a **Gaussian Naive Bayes** classifier written from scratch in **NumPy**, including the class means, variances, priors and likelihood function.
- Converted the images to **HSV** with **OpenCV** and trained on the hue of every pixel across 31 labelled scenes, separating road from lane markings, signs, cars and background.

Tech Company Acquisition Price Prediction [↗](#)

- Predicted what an acquirer would pay for a tech company, joining four raw **CSV** sources (acquired

firms, acquirers, deals and founders) into one feature table in **pandas**, and comparing **Gradient Boosting**, **Random Forest**, **XGBoost** and linear models in **scikit-learn** on both the price and the deal-size class.

- Engineered the features the raw data did not have: company age at acquisition, one-hot expansion of multi-valued market categories and deal terms, grouping of rare countries, and repair of mistyped years found by plotting the data.
- Handled the messy numerics with a **Shapiro-Wilk** normality test, **IQR** outlier capping and log transforms on skewed columns, then **KNN imputation** for the gaps, and tuned each model with grid and randomized search.

Eye-Movement Control from Eye Signals [↗](#)

- Turned eye movements into computer commands, classifying 181 recordings of the electrical signal around the eye into five gestures (up, down, left, right and blink) after a **SciPy** pipeline of DC removal, a 4th-order **Butterworth** bandpass and z-score normalization.
- Compared three representations of the same signal (raw samples, **autoregressive** coefficients, **db4 wavelet** energies, and a fusion of the last two), training an **SVM** on each with **GridSearchCV** over kernel, C and gamma.

Meal Recognition and Calorie Estimation [↗](#)

- Estimated the calories in a meal photo in **PyTorch**, chaining five trained models behind one pipeline: an **EfficientNet** that sorts the photo into food or fruit, then a **ResNet** that names the fruit or a **Siamese** network that names the dish.
- Handled unseen dishes with **one-shot** recognition rather than a fixed label set, matching a query against reference embeddings by distance and returning no match when nothing is close enough, so new dishes can be added from a single example without retraining.
- Segmented the fruit from its background with a **U-Net** at **0.99 IoU**, and added a 31-class segmentation model in **TensorFlow** to label each fruit type pixel by pixel.

Interior Design Style Classification [↗](#)

- Sorted photos of rooms into 17 interior-design styles for a **Kaggle** competition, rebuilding **Vision Transformer**, **MobileNetV2**, **GoogLeNet** and **VGG16** from their papers in **PyTorch** (patch embedding, multi-head self-attention, inverted residuals, inception blocks) and benchmarking each against its pretrained equivalent.
- Countered a class-imbalanced training set with a **weighted sampler** and online augmentation, and shipped a standalone inference script per architecture.

Encryption and Cryptanalysis Toolkit [↗](#)

- Implemented ten ciphers from scratch in **Java** with no cryptography libraries, from classical ones (Caesar, Playfair, Hill, Vigenère, rail fence, columnar transposition) up to full **AES-128** and **DES** block ciphers, including the byte substitution, row shifts, column mixing and key expansion.
- Broke the classical ciphers as well as building them, recovering the key from a known plaintext and ciphertext pair, and cracking the monoalphabetic cipher by letter-frequency analysis alone, all covered by **45 JUnit tests**.

Pac-Man with Pathfinding Ghosts [↗](#)

- Built a Pac-Man clone in **C++** and **raylib** with four ghosts that each run a **breadth-first search** over the maze grid and step along the shortest path; difficulty changes what they search for, from a random corridor to Pac-Man himself.
- Loaded the mazes from **CSV** files and shipped a mouse-driven map editor that lets a player draw a maze and immediately play it, with persisted high scores and a resizable window that rescales the grid.

Graph-Based Image Segmentation [🔗](#)

- Split images into regions in **C#**, implementing a **graph segmentation** algorithm from its paper over a graph built from every pixel.
- Sorted those edges in **linear time** with a 256-bucket **counting sort** rather than an $O(E \log E)$ comparison sort.
- Segmented the red, green and blue channels independently and intersected the three results.

Technical Skills

Languages: Python, C/C++, C#, Java, PHP, JavaScript, SQL, Bash

Tools/Frameworks: PyTorch, TensorFlow, scikit-learn, NumPy, Pandas, SciPy, OpenCV, LangChain, FastAPI, Laravel, Docker, Git, Linux, Django

Training & Development

- **MEC Model of Leadership:** Completed an extensive leadership program (160 hours) covering emotional intelligence, effective communication, self-awareness (50 hours), the Enneagram, problem-solving, and team building.